L36 — Bellman-Ford Algorithm

Unit: 3 | **Chapter:** Graphs | **BT Level:** BT5 | **CO:** CO3, CO4

Learning Objectives

- Explain why Bellman-Ford handles negative-weight edges where Dijkstra fails
- Implement Bellman-Ford and trace its relaxation steps
- Detect negative-weight cycles using Bellman-Ford
- Compare Bellman-Ford and Dijkstra in terms of scope and complexity

---

1. Motivation: Dijkstra's Limitation

Dijkstra's algorithm uses a greedy approach that relies on **non-negative edge weights**. When negative edges exist, Dijkstra can finalize a vertex too early with an incorrect distance.

Bellman-Ford handles negative-weight edges by relaxing all edges **V-1 times** (iteratively), guaranteeing convergence.

---

2. Key Idea

Observation: In a graph with V vertices, the shortest path between any two vertices (with no negative cycles) has at most **V-1 edges**.

Algorithm: Relax all edges V-1 times. After the k-th iteration, all shortest paths using at most k edges are correctly computed.

---

3. Algorithm

```
Algorithm BellmanFord(graph, source):
  dist[v] = ∞ for all v
  dist[source] = 0
  parent[v] = None for all v

  // V−1 relaxation rounds
  for i from 1 to V−1:
      for each edge (u, v, weight) in graph:
          if dist[u] + weight < dist[v]:
              dist[v] = dist[u] + weight
              parent[v] = u

  // Detect negative cycle: if any edge can still be relaxed, cycle exi
  for each edge (u, v, weight) in graph:
      if dist[u] + weight < dist[v]:
          raise NegativeCycleError("Negative cycle detected")
```

```
  return dist, parent
```

---

4. Implementation

```
def bellman_ford(vertices, edges, source):
    """
    vertices: list of vertex names
    edges: list of (u, v, weight) tuples
    source: starting vertex

    Returns (dist, parent) or raises ValueError if negative cycle exist
    """
    dist = {v: float('inf') for v in vertices}
    dist[source] = 0
    parent = {v: None for v in vertices}

    V = len(vertices)

    # V-1 relaxation rounds
    for _ in range(V - 1):
        updated = False
        for u, v, w in edges:
            if dist[u] != float('inf') and dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                parent[v] = u
                updated = True
        if not updated:   # Early termination if no updates
            break

    # Check for negative cycles
    for u, v, w in edges:
        if dist[u] != float('inf') and dist[u] + w < dist[v]:
            raise ValueError(f"Negative weight cycle detected involving

    return dist, parent


def reconstruct_path(parent, source, target):
    """Trace back from target to source using parent pointers."""
    path = []
    node = target
    while node is not None:
        path.append(node)
        node = parent[node]
        if node == target:   # Detect cycle in path (shouldn't happen i
            break
    path.reverse()
    if path[0] != source:
        return None   # Target not reachable
    return path
```

5. Detailed Trace

Graph:

```
A −(6)→ B
A −(7)→ C
B −(5)→ C   B −(−4)→ D
C −(−3)→ B   C −(9)→ D
D −(2)→ A   D −(7)→ B
```

Vertices: {A, B, C, D, E}, Source: A

Edges: (A,B,6), (A,C,7), (B,C,5), (B,D,-4), (C,B,-3), (C,D,9), (D,A,2), (D,B,7)

Initial: `dist = {A:0, B:∞, C:∞, D:∞}`

Round 1 (relax all edges):

- (A,B,6): dist[B] = 0+6 = 6 ✓
- (A,C,7): dist[C] = 0+7 = 7 ✓
- (B,C,5): dist[C] = min(7, 6+5=11) → no update
- (B,D,-4): dist[D] = 6+(-4) = 2 ✓
- (C,B,-3): dist[B] = min(6, 7+(-3)=4) = 4 ✓
- (C,D,9): dist[D] = min(2, 7+9=16) → no update
- (D,A,2): dist[A] = min(0, 2+2=4) → no update
- (D,B,7): dist[B] = min(4, 2+7=9) → no update

After Round 1: `{A:0, B:4, C:7, D:2}`

Round 2:

- (A,B,6): dist[B] = min(4, 0+6) → no update
- (B,C,5): dist[C] = min(7, 4+5=9) → no update
- (B,D,-4): dist[D] = min(2, 4-4=0) = 0 ✓
- (C,B,-3): dist[B] = min(4, 7-3=4) → no update
- (D,B,7): dist[B] = min(4, 0+7=7) → no update

After Round 2: `{A:0, B:4, C:7, D:0}`

Round 3 — no updates: Algorithm converges.

Final distances from A: A=0, B=4, C=7, D=0

---

6. Negative Cycle Detection

```
def has_negative_cycle(vertices, edges, source):
    """Return True if graph has a negative cycle reachable from source.
    dist = {v: float('inf') for v in vertices}
    dist[source] = 0

    for _ in range(len(vertices) - 1):
        for u, v, w in edges:
```

```
            if dist[u] != float('inf') and dist[u] + w < dist[v]:
                dist[v] = dist[u] + w

    # V-th round: if any relaxation still possible → negative cycle
    for u, v, w in edges:
        if dist[u] != float('inf') and dist[u] + w < dist[v]:
            return True

    return False
```

Why does the V-th round detect cycles?
After V-1 rounds, all shortest paths are finalized (they have at most V-1 edges). If a V-th round still reduces a distance, it means there's a cycle of negative total weight that can be traversed indefinitely.

---

7. Bellman-Ford vs Dijkstra

Feature	Dijkstra	Bellman-Ford
Handles negative edges	No	Yes
Detects negative cycles	No	Yes
Time complexity	O((V + E) log V)	O(VE)
Space complexity	O(V)	O(V)
Algorithm type	Greedy	Dynamic Programming
Implementation	Priority queue	Iterate all edges V-1 times
Use case	GPS routing (non-negative weights)	Financial arbitrage, general SSSP

---

8. Application: Currency Arbitrage

Currencies are vertices; exchange rates are edges.
Transform: **weight = -log(exchange_rate)**
Then a negative cycle = a sequence of exchanges that yields profit.

```
import math

# Exchange rates
rates = {
    ('USD', 'EUR'): 0.85,
    ('EUR', 'GBP'): 0.90,
    ('GBP', 'USD'): 1.35,
}

# Convert to negative log
log_edges = [(u, v, -math.log(r)) for (u, v), r in rates.items()]

# If Bellman-Ford detects a negative cycle → arbitrage opportunity
```

Summary

- Bellman-Ford: relaxes all edges V-1 times → O(VE) time.
- Handles **negative-weight edges** (Dijkstra cannot).
- Detects **negative-weight cycles** in a V-th relaxation round.
- Correctness: after k rounds, all shortest paths with $\leq$ k edges are correct.
- Use Dijkstra for efficiency when weights are non-negative; Bellman-Ford when negative edges or cycle detection is needed.

References

- Cormen et al., *Introduction to Algorithms*, Ch. 24.1 (MIT Press, 2022)
- Skiena, *The Algorithm Design Manual*, Ch. 8.2 (Springer, 2020)